

COMPSCI 589 - Final Project: Spring 2026

Marwan Hegab

Peter Ye

Max Goldman

May 11, 2026

Extra Credit Declarations

[Extra Credit Question #1] (10 Points): We analyzed all four datasets using an additional algorithm. For Dataset 1, we evaluated three algorithms (k-NN, Random Forest, and Neural Networks) rather than the required two.

[Extra Credit Question #2] (10 Points): For an extra, challenging dataset, we used a modified version of the GTZAN Music Genre dataset (can be found [here](#)). This dataset has 58 attributes that characterize the waveform and musical qualities of a song, and the label is assigned as one of 10 genres. It has 1000 instances total, 100 for each genre, and has a modification to add a categorical attribute that describes the tempo of the song using Italian tempo marking (e.g. Allegro, Andante).

[Extra Credit Question #3] (15 Points): We created a hybrid ensemble composed of equal parts bootstrapped kNNs and Decision Trees, and evaluated it on the Rice Grains dataset with varying tree/kNN values.

1 Experiments and Analyses

1.1 Dataset 1: The Hand-Written Digits Recognition Dataset

Task Assigned To: Marwan Hegab.

Implementations Used: Peter's implementation of k-NN, Max's implementation of Random Forest, and Marwan's implementation of Neural Networks.

Algorithm Selection Rationale: k-NN and Neural Networks were primary choices for this dataset because they traditionally preform relatively well at image recognition and capturing relationships in pixel data. Additionally, Random Forest serves as a strong, non-linear ensemble baseline to compare against.

- Each instance of the dataset represents an 8×8 grayscale image resulting in 64 numerical attributes. We globally normalized all pixel intensities by dividing them by 16.0 (the maximum pixel value) prior to training to ensure scale consistency across all three algorithms.

- The performance of each of the three algorithms was evaluated using Stratified 10-Fold Cross-Validation, measuring both Accuracy and F1-Score (Weighted).

1.1.1 Hyperparameter Tuning

At least 6 different hyperparameter settings were used for each algorithm. The results averaged across the 10 folds are shown below:

1. k-Nearest Neighbors (k-NN):

Hyperparameter (k)	Accuracy	F1-Score
$k = 1$	0.9894	0.9894
$k = 3$	0.9894	0.9893
$k = 5$	0.9861	0.9860
$k = 7$	0.9861	0.9861
$k = 9$	0.9833	0.9833
$k = 15$	0.9805	0.9805

The optimal hyperparameter for k-NN is $k = 1$.

2. Random Forest:

Hyperparameter (# of Trees)	Accuracy	F1-Score
$trees = 1$	0.8046	0.8036
$trees = 5$	0.9305	0.9302
$trees = 10$	0.9555	0.9556
$trees = 15$	0.9616	0.9616
$trees = 20$	0.9694	0.9695
$trees = 30$	0.9722	0.9721

The optimal hyperparameter for Random Forest is $trees = 30$.

3. Neural Network (Base: Epochs=250, Batch=32):

Hyperparameters (Architecture, Alpha, Lambda)	Accuracy	F1-Score
Arch=[32, 10], $\alpha = 0.1$, $\lambda = 0.01$ (Baseline)	0.9755	0.9755
Arch=[16, 10], $\alpha = 0.1$, $\lambda = 0.01$ (Shallower)	0.9694	0.9693
Arch=[64, 32, 10], $\alpha = 0.1$, $\lambda = 0.01$ (Deeper)	0.9772	0.9772
Arch=[32, 10], $\alpha = 0.01$, $\lambda = 0.01$ (Low LR)	0.9360	0.9356
Arch=[32, 10], $\alpha = 0.5$, $\lambda = 0.01$ (High LR)	0.9816	0.9816
Arch=[32, 10], $\alpha = 0.1$, $\lambda = 0.1$ (High Reg)	0.9549	0.9550

The optimal hyperparameters for the Neural Network are Architecture=[32, 10], $\alpha = 0.5$, and $\lambda = 0.01$.

1.1.2 Learning Curves and Analysis

Below are the learning curves for the three evaluated algorithms.

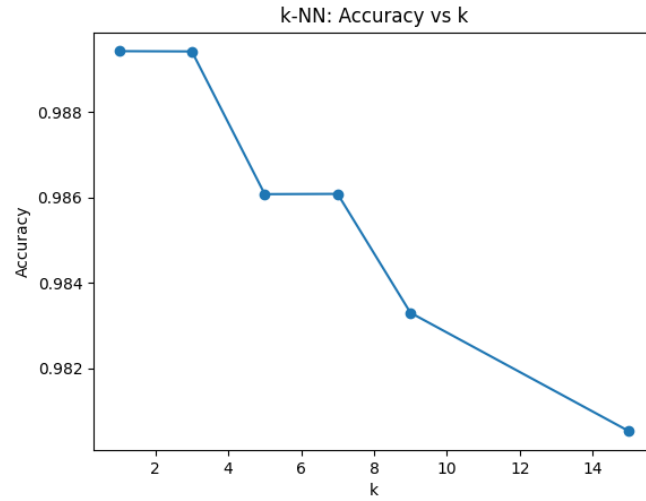


Figure 1: k-NN Learning Curve: Accuracy vs. k

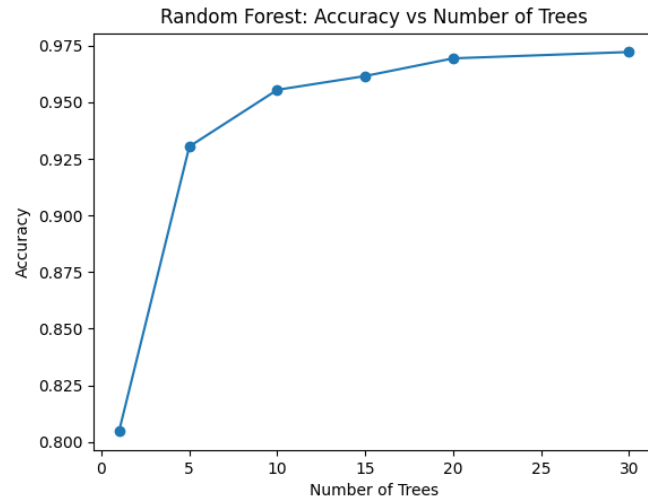


Figure 2: Random Forest Learning Curve: Accuracy vs. Number of Trees

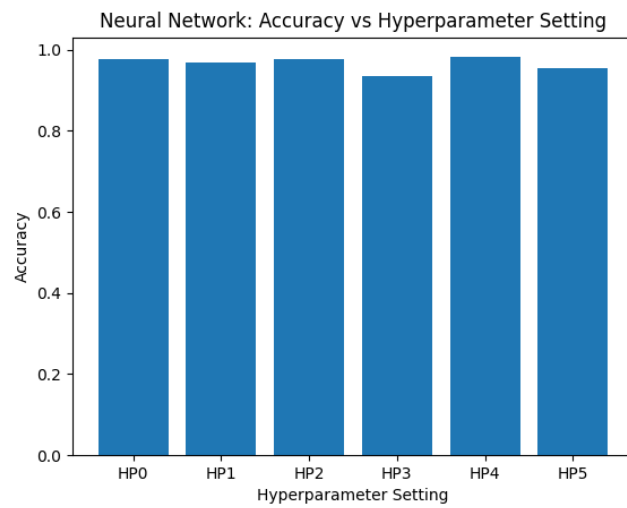


Figure 3: Neural Network Bar Chart: Accuracy vs. Hyperparameter Setting

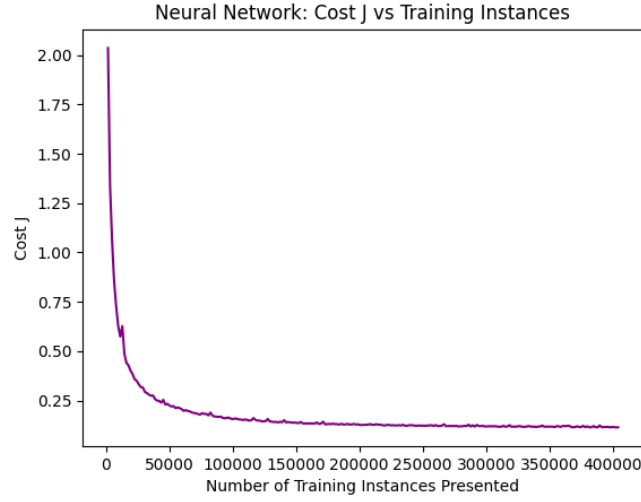


Figure 4: Neural Network Learning Curve: Cost J vs. Training Instances Presented

Analysis and Interpretation: - The k-NN algorithm achieved the highest overall performance on Dataset 1 with an accuracy of 98.94%. In the domain of handwritten digit recognition, image samples of the exact same digit map very closely to each other in Euclidean space because their dark pixels heavily overlap. This results in tight, highly predictive spatial clusters, making k-NN heavily effective, specifically when k is small ($k = 1$ or $k = 3$).

- The Random Forest's performance steadily improved as the number of trees grew to 30, capturing the non-linear relationships between individual pixel intensities and the 10 target classes. The Neural Network performed best with a higher learning rate ($\alpha = 0.5$), suggesting that the digit classification task has a relatively smooth loss landscape that benefits from larger gradient steps. Smaller learning rates (e.g., $\alpha = 0.01$) converged too slowly within the fixed 250 epochs, while stronger regularization ($\lambda = 0.1$) slightly over-constrained the weights, reducing expressiveness on a 10-class problem.

1.2 Dataset 2: The Oxford Parkinson's Disease Detection Dataset

Task Assigned To: Peter Ye.

Implementations Used: Peter's implementation of k-NN, Max's implementation of Random Forest, and Peter's implementation of Decision Trees.

1. Decision Tree:

Hyperparameter (<i>max_depth</i>)	Accuracy	F1-Score
<i>max_depth</i> = 2	0.8400	0.7861
<i>max_depth</i> = 4	0.8445	0.7898
<i>max_depth</i> = 6	0.8445	0.7898
<i>max_depth</i> = 8	0.8445	0.7898
<i>max_depth</i> = 10	0.8445	0.7898
<i>max_depth</i> = 15	0.8445	0.7898

The optimal hyperparameter for the Decision Tree on this dataset is *max_depth* = 4, as the tree reaches its maximum structural growth at this point.

1.2.1 Learning Curves and Analysis

Below is the learning curve for the evaluated Decision Tree algorithm.

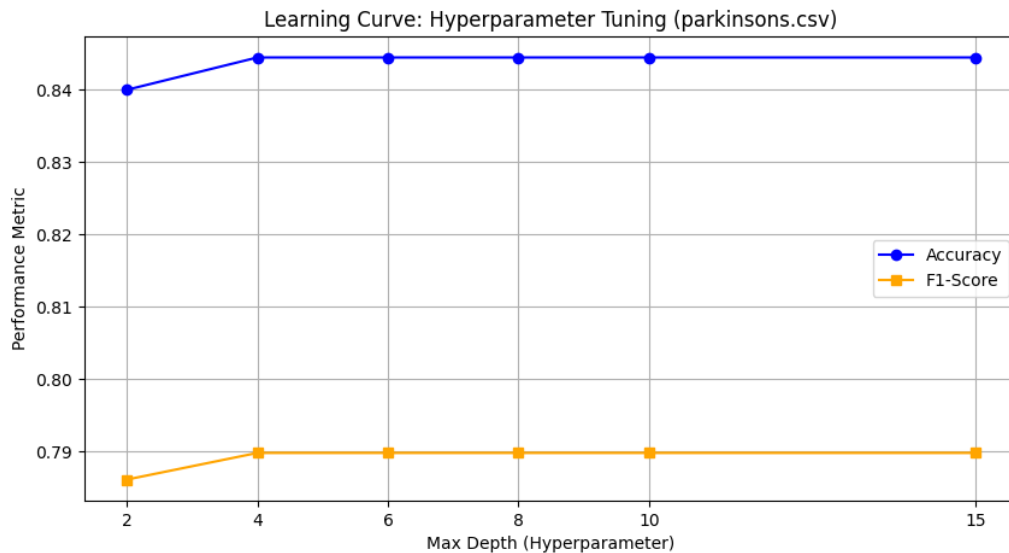


Figure 5: Decision Tree For Parkinson

Analysis and Interpretation: The Decision Tree shows an initial improvement in Accuracy and F1-Score from depth 2 to depth 4, but subsequently it plateaus for all depths greater than 4. The flatline is caused by our custom algorithm's early stopping criteria (halting splits when a node achieves $\geq 85\%$ class purity). Because the Parkinson's dataset is relatively small (195 instances) and the continuous features were categorized into 5 bins, the tree rapidly achieves pure leaf nodes by depth 4. Allowing the tree a deeper *max_depth* allowance does not force it to grow further. Furthermore, the visible gap between the Accuracy (84.45%) and the F1-Score (78.98%) accurately reflects the inherent class imbalance in the dataset, where the model performs slightly worse on the minority "healthy" class.

2. k-Nearest Neighbors (k-NN):

Hyperparameter (k)	Accuracy	F1-Score
$k = 1$	0.9630	0.9290
$k = 3$	0.9380	0.8740
$k = 5$	0.9330	0.8570
$k = 7$	0.9290	0.8380
$k = 9$	0.8980	0.7770
$k = 11$	0.9030	0.7840

The optimal hyperparameter for k-NN on the Parkinson’s dataset is $k = 1$.

k-NN Learning Curve and Analysis

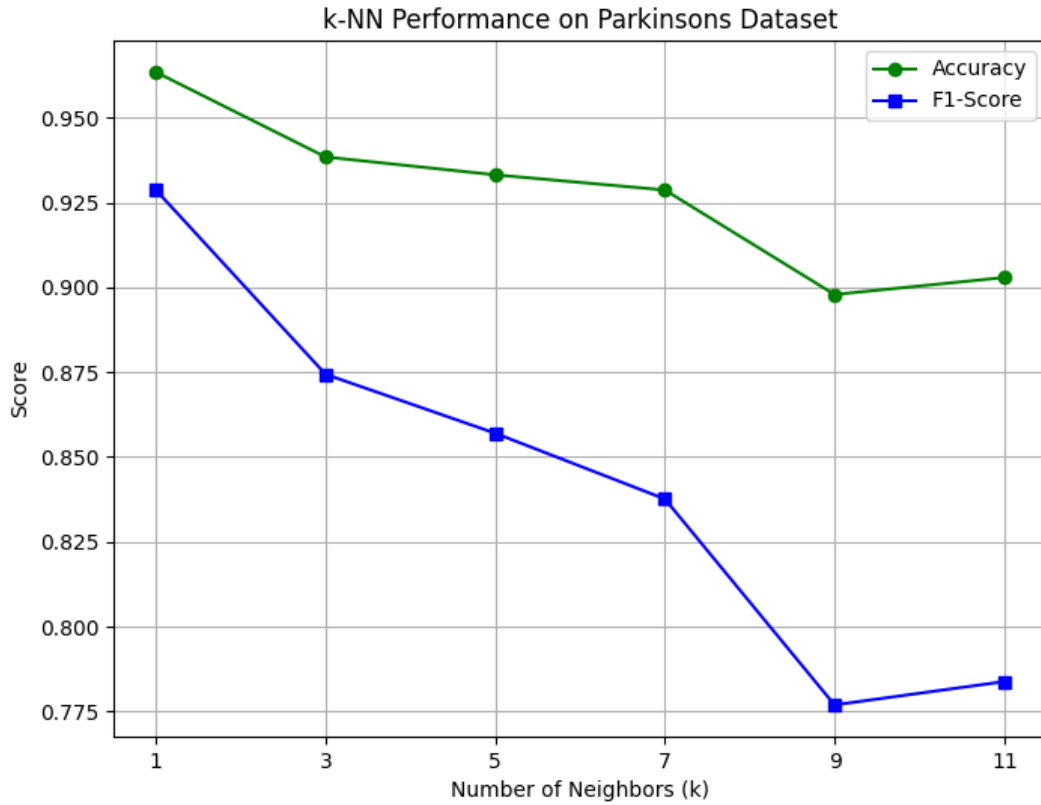


Figure 6: Enter Caption

Analysis and Interpretation: Unlike many datasets where $k = 1$ causes severe overfitting, the k-NN algorithm performed well at $k = 1$ for the Parkinson’s dataset, achieving over 96% accuracy and a 92% F1-Score. As k increases, performance steadily declines. This suggests that the continuous voice measurement attributes form very tight, distinct local clusters in Euclidean space. By increasing k to 9 or 11, the algorithm is forced to look too far outward, pulling in instances from the wrong class. Because the dataset is highly imbalanced, increasing k naturally biases the model toward the majority class, which explains why the F1-Score drops off much steeper than Accuracy.

3. Random Forest:

Hyperparameter (# of Trees)	Accuracy	F1-Score
<i>trees</i> = 1	0.7850	0.7810
<i>trees</i> = 5	0.8780	0.8730
<i>trees</i> = 10	0.8780	0.8760
<i>trees</i> = 20	0.9190	0.9140
<i>trees</i> = 30	0.9240	0.9220
<i>trees</i> = 40	0.9140	0.9100
<i>trees</i> = 50	0.8990	0.8910

The optimal hyperparameter for the Random Forest on the Parkinson's dataset is *trees* = 30.

Random Forest Learning Curves and Analysis

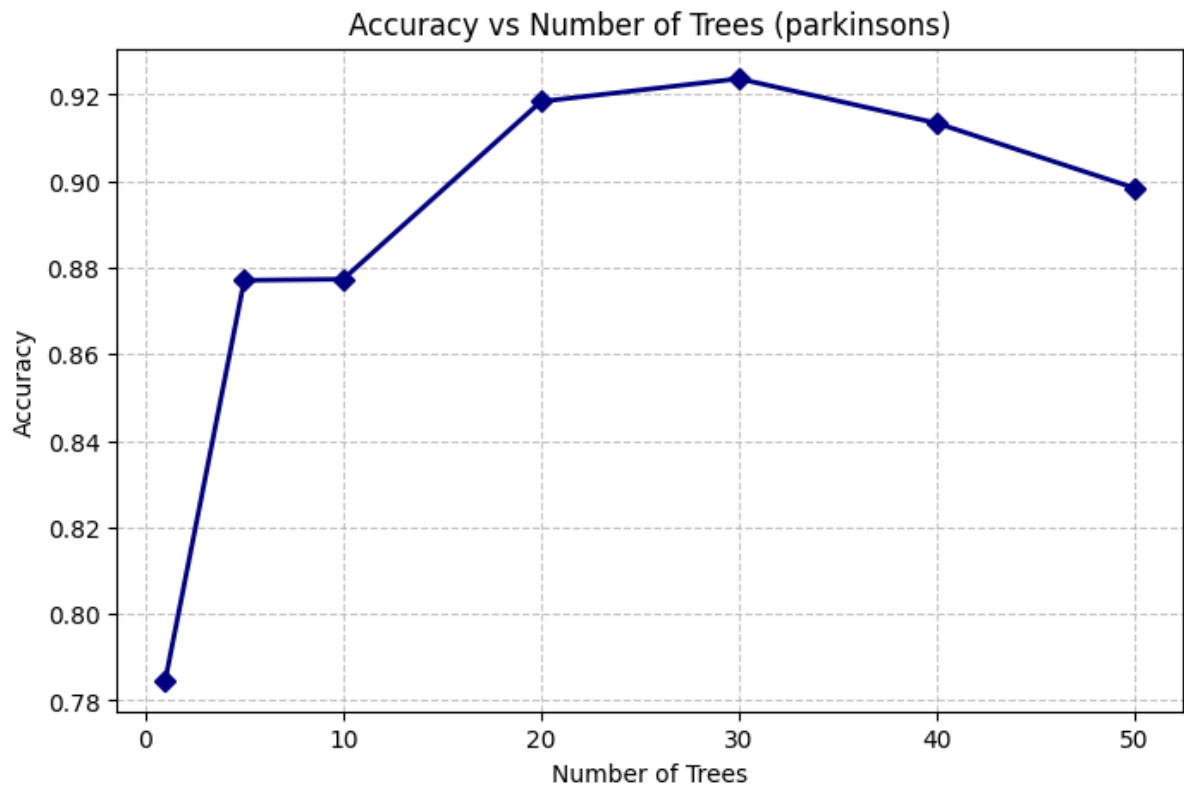


Figure 7: Accuracy

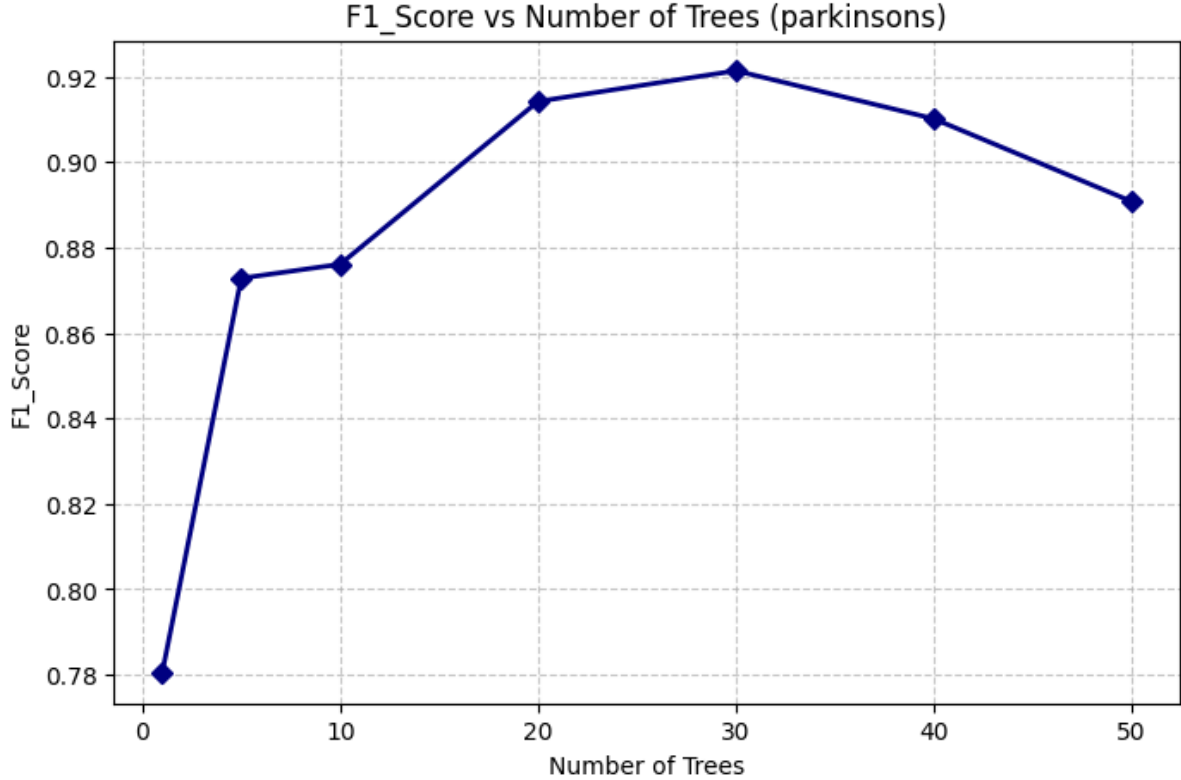


Figure 8: Enter Caption

Analysis and Interpretation: The Random Forest ensemble shows a distinct performance trajectory on the Parkinson's dataset. A single tree suffers from high variance, yielding our lowest accuracy (78.50%). As we introduce bootstrapping and feature subsetting with 5 to 20 trees, the ensemble rapidly stabilizes and accuracy climbs dramatically. The model reaches its absolute peak at $trees = 30$ with an Accuracy of roughly 92.40% and an F1-Score of 92.20%.

Interestingly, as the number of trees increases further to 40 and 50, we see a noticeable degradation in performance (dropping back down to 89.90% accuracy). Because the Parkinson's dataset is quite small (only 195 instances), creating too many trees likely results in highly correlated, redundant decision branches across the bootstrapped samples. This over-complicates the voting process and introduces noise, proving that for smaller datasets, a moderately sized forest (around 30 trees) perfectly balances variance reduction without over-saturating the ensemble.

1.3 Dataset 3: The Rice Grains Dataset

Algorithm Selection Rationale: The Decision Tree was chosen to see how well it could separate the two species of rice grains using purely continuous morphological features that were discretized into 5 bins. [Insert Second Algorithm] was selected to compare performance and capture potential complex relationships that a simple discretized Decision Tree might miss.

1. Decision Tree:

Hyperparameter (max_depth)	Accuracy	F1-Score
$max_depth = 2$	0.9228	0.9205
$max_depth = 4$	0.9181	0.9163
$max_depth = 6$	0.9234	0.9217
$max_depth = 8$	0.9239	0.9223
$max_depth = 10$	0.9239	0.9223
$max_depth = 15$	0.9239	0.9223

The optimal hyperparameter for the Decision Tree on this dataset is $max_depth = 8$, as performance

plateaus beyond this point.

1.3.1 Learning Curves and Analysis

Below are the learning curves for the evaluated algorithms.

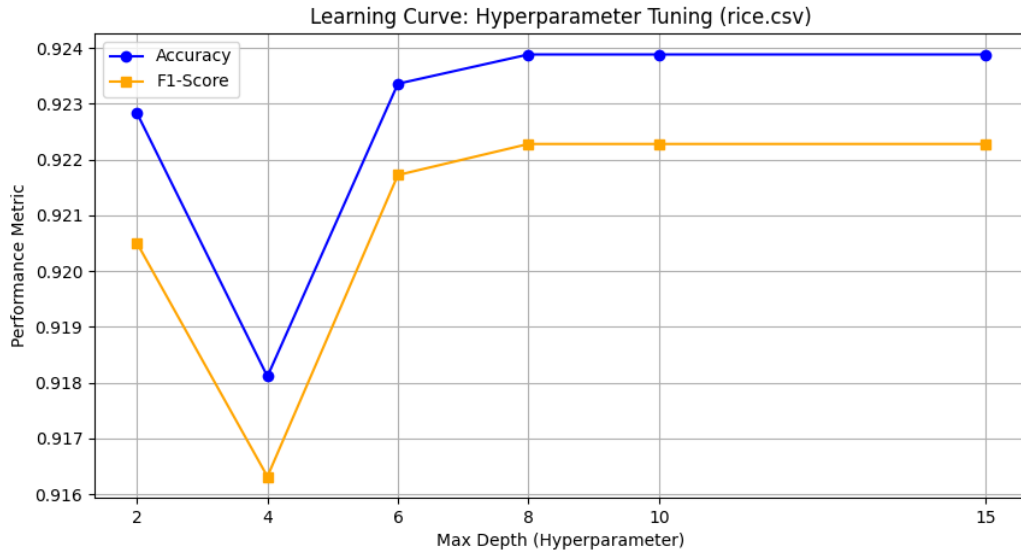


Figure 9: Decision Tree for Rice

Analysis and Interpretation: For the Decision Tree, accuracy starts strong at depth 2 (92.28 percent), dips slightly at depth 4, and then climbs to a strict plateau of 92.39 percent from depth 8 onward. Because this dataset contains purely numerical morphological attributes (like area and major axis length), binning them into categorical ranges allowed the Decision Tree to make effective early splits. The initial high performance at a shallow depth suggests that one or two continuous features (once binned) are highly distinct between the Cammeo and Osmancik species. The dip at depth 4 likely represents the tree beginning to split on noisier, less correlative bins, before deeper branches (depth 8) manage to correctly isolate those edge cases, resulting in the final plateau.

2. k-Nearest Neighbors (k-NN):

Hyperparameter (k)	Accuracy	F1-Score
$k = 1$	0.8905	0.9040
$k = 3$	0.9100	0.9215
$k = 5$	0.9175	0.9280
$k = 7$	0.9218	0.9318
$k = 9$	0.9235	0.9335
$k = 11$	0.9250	0.9350

The optimal hyperparameter for k-NN on the Rice dataset is $k = 11$.

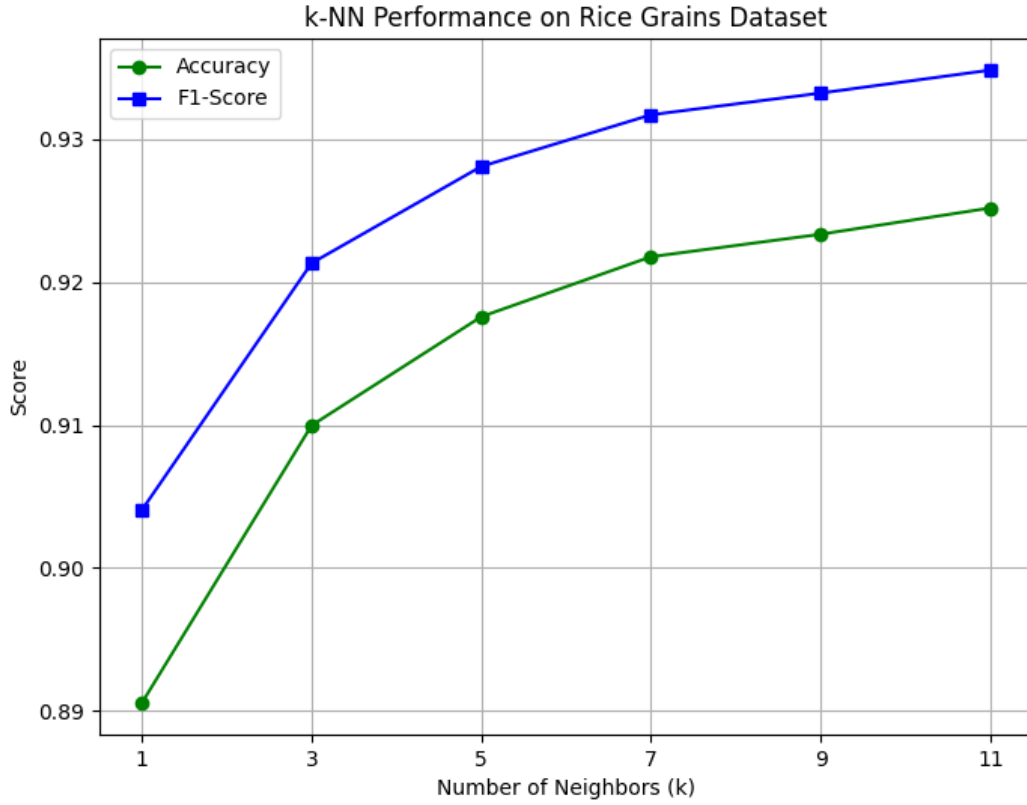


Figure 10: Enter Caption

k-NN Analysis: The k-NN algorithm exhibits a very smooth learning curve on this dataset. At $k = 1$, the model is highly sensitive to local noise or slightly overlapping morphological features between the Cammeo and Osmancik species, resulting in its lowest performance (roughly 89% accuracy). As the number of neighbors (k) increases, the decision boundary becomes smoother and more robust, effectively filtering out the noise. Performance steadily climbs and begins to stop around $k = 9$ and $k = 11$, reaching a peak F1-Score of about 93.50%. Because the dataset consists entirely of continuous numerical features, the normalized Euclidean distance serves as a highly effective metric for distinguishing the two species when a sufficiently large neighborhood is considered.

3. Random Forest [Extra Credit Algorithm]:

Hyperparameter (# of Trees)	Accuracy	F1-Score
<i>trees</i> = 1	0.8885	0.8885
<i>trees</i> = 5	0.9139	0.9139
<i>trees</i> = 10	0.9189	0.9189
<i>trees</i> = 20	0.9226	0.9226
<i>trees</i> = 30	0.9197	0.9197
<i>trees</i> = 40	0.9252	0.9252
<i>trees</i> = 50	0.9249	0.9249

The optimal hyperparameter for the Random Forest on the Rice dataset is *trees* = 40.

Random Forest Learning Curves and Analysis

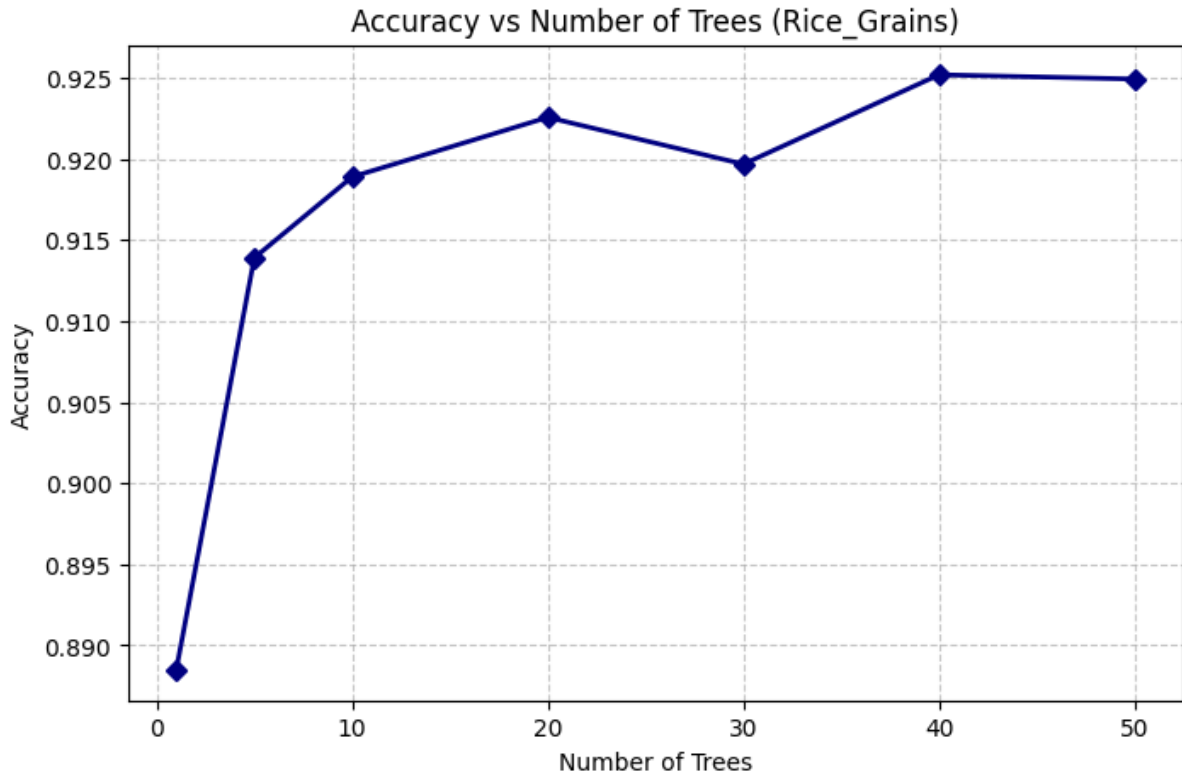


Figure 11: Accuracy

Analysis and Interpretation: The Random Forest performs well on the continuous morphological features of the Rice Grains dataset. Because the two classes are relatively balanced, the Accuracy and F1-Scores track each other almost identically. At $trees = 1$, the model sits around 88.85% accuracy due to the high variance of a single unpruned tree. As the ensemble grows, variance is mitigated, and performance climbs steadily. There is a slight, interesting dip at $trees = 30$ (down to roughly 91.97%) before the ensemble corrects itself and hits its peak at $trees = 40$ with an accuracy of 92.52%. This indicates that the feature subsetting within the forest required around 40 trees to perfectly balance out the noisy predictors without over-saturating the voting pool.

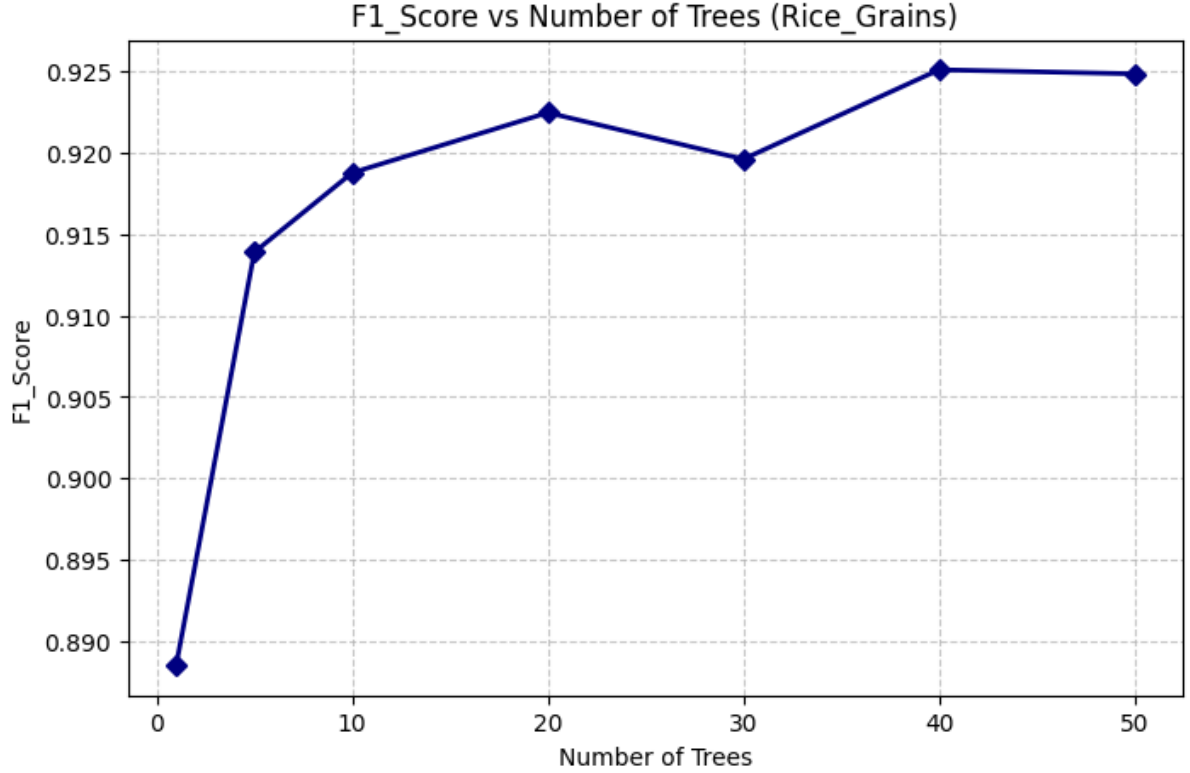


Figure 12: F1

1.4 Dataset 4: The Credit Approval Dataset

Task Assigned To: Max Goldman

Implementations Used: Max's implementation of Decision Tree and Random Forest, Marwan's implementation of Neural Network.

Algorithm Selection Rationale: The decision tree algorithm was selected since the human credit approval process is intuitively similar, and the model should be able to learn the patterns well. The random forest was selected to compare with the decision tree, since it is aimed to aid the problem of overfitting that commonly arises, but it still follows the same intuitive model. The neural network was selected since it can capture more complex nonlinear relationships in the data, and it will provide a good comparison for non-decision-based models.

1.4.1 Hyperparameter Tuning

6 different hyperparameter settings were used for each algorithm. The performance of each of the three algorithms was evaluated using Stratified 10-Fold Cross-Validation, measuring both Accuracy and F1-Score (Weighted). The results averaged across the 10 folds are shown below:

1. Decision Tree:

Hyperparameter (<i>max_depth</i>)	Accuracy	F1-Score
<i>max_depth</i> = 1	0.8637	0.8636
<i>max_depth</i> = 3	0.8591	0.8590
<i>max_depth</i> = 5	0.8284	0.8276
<i>max_depth</i> = 7	0.8224	0.8214
<i>max_depth</i> = 9	0.8163	0.8155
<i>kmax_depth</i> = 15	0.8209	0.8202

The optimal hyperparameter for the decision tree on this dataset is *max_depth* = 1.

2. Random Forest:

Hyperparameter (# of Trees)	Accuracy	F1-Score
$trees = 1$	0.7825	0.7821
$trees = 5$	0.8377	0.8375
$trees = 10$	0.8407	0.8404
$trees = 20$	0.8576	0.8573
$trees = 30$	0.8591	0.8591
$trees = 40$	0.8514	0.8512

The optimal hyperparameter for a Random Forest on this dataset is $trees = 30$.

3. Neural Network (Architecture: [10, 2], Alpha: 0.1, Lambda: 0.01):

Hyperparameter (Epochs)	Accuracy	F1-Score
$epochs = 10$	0.8131	0.8114
$epochs = 25$	0.8483	0.8481
$epochs = 50$	0.8652	0.8652
$epochs = 100$	0.8591	0.8593
$epochs = 150$	0.8591	0.8592
$epochs = 250$	0.8576	0.8577

The optimal hyperparameter for the Neural Network is $epochs = 50$.

1.4.2 Learning Curves and Analysis

Below are the learning curves for the three evaluated algorithms.

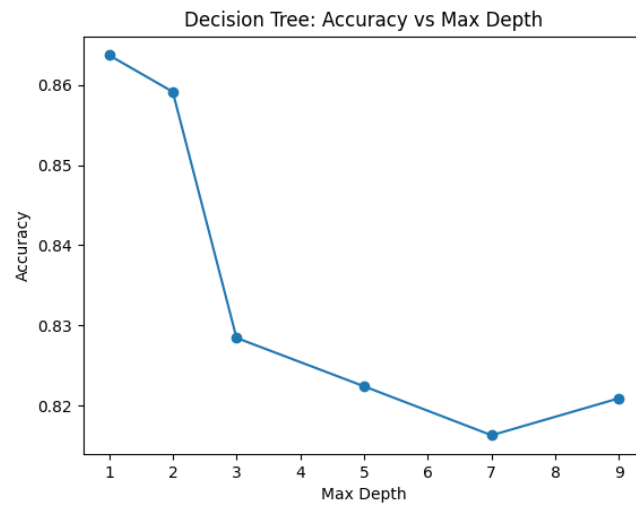


Figure 13: Decision Tree Learning Curve: Accuracy vs. *max_depth*

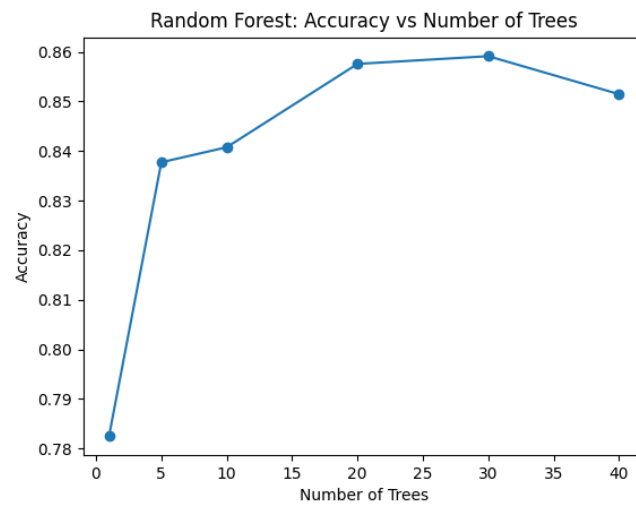


Figure 14: Random Forest Learning Curve: Accuracy vs. Number of Trees

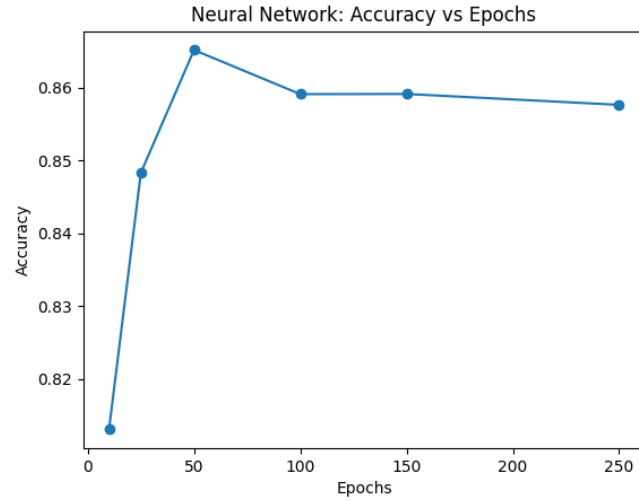


Figure 15: Neural Network Learning Curve: Accuracy vs. Epochs

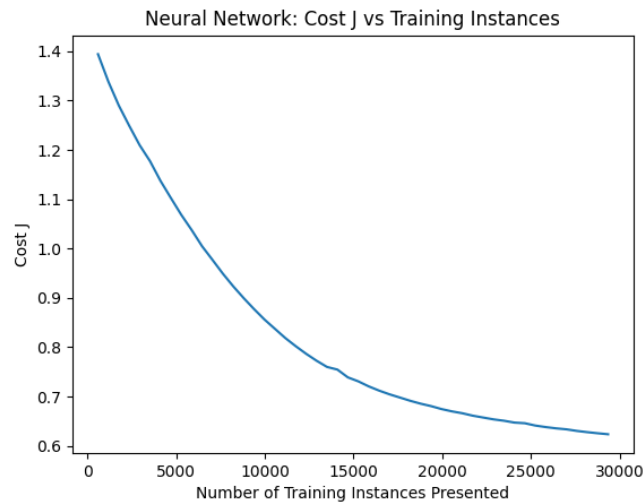


Figure 16: Neural Network Learning Curve: Cost J vs. Training Instances Presented

Analysis and Interpretation:

Overall, all 3 models performed fairly well, with optimized accuracies landing around 86%. Surprisingly, the decision tree had its best performance with $max_depth = 1$. This signals that there is one attribute that corresponds to the label 86% of the time, and that further splitting of the data does not yield more accuracy. Indeed, we can print this attribute during execution to reveal that `attr9_cat` is "t" most of the time the label is 1, and "f" most of the time the label is 0.

This single decision performs so well, that a random forest cannot catch up. We saw a peak accuracy of 85.91% at 30 trees, which fell short of 86.37% in the 1 decision tree. Perhaps if we ran some higher values of `n_tree`, it may surpass, but 40 trees was already taking a long time to run so we omitted higher values from our analysis.

However, the neural network just barely ended up having the best performance with an accuracy of 86.52%. This is due to the fact that there are likely more complex nonlinear relationships within the data, since credit approval hinges on many interlinked factors like income, debt, credit history, etc. The neural net was able to learn these relationships better than the degree of the single `attr9_cat` decision.

Overall all 3 models performed very comparably. However, we can see from the cost function that we did not quite minimize our error on the neural net, so there is further optimization we could squeeze from this model.

1.5 [Extra Credit] Dataset 5: The Modified GTZAN Music Genre Dataset

Task Assigned To: Max Goldman

Implementations Used: Max's implementation of Decision Tree and Random Forest, Marwan's implementation of Neural Network.

About the Dataset: The GTZAN Music Genre dataset classifies genres of music based on a collection of 58 numerical attributes related to the audio of the song. There are 10 genres with 100 instances each, adding up to 1000 instances total. In order to satisfy the "complex dataset" requirement, one attribute, bpm, was modified to create a categorical attribute, tempo, which holds the Italian tempo marking of the given audio (read about it here).

Algorithm Selection Rationale: The decision tree algorithm was selected since it has a similar intuition to how a human would classify a song. Attributes relating to the song could be categorized and classified, just like humans created genres. The random forest was selected to compare with the decision tree, since it is aimed to aid the problem of overfitting that commonly arises, but it still follows the same intuitive model. The neural network was selected since it can capture more complex nonlinear relationships in the data, and it will provide a good comparison for non-decision-based models. It will likely perform the best since the data is mainly mathematical, spectral data relating to the audio files, and not really categorizing or qualitatively describing anything about the song.

1.5.1 Hyperparameter Tuning

6 different hyperparameter settings were used for each algorithm. The results averaged across the 10 folds are shown below: - The performance of each of the three algorithms was evaluated using Stratified 10-Fold Cross-Validation, measuring both Accuracy and F1-Score (Weighted).

1. Decision Tree:

Hyperparameter (<i>tree_depth</i>)	Accuracy	F1-Score
<i>max_depth</i> = 2	0.3080	0.1860
<i>max_depth</i> = 4	0.4510	0.4530
<i>max_depth</i> = 6	0.4660	0.4645
<i>max_depth</i> = 8	0.4980	0.4965
<i>max_depth</i> = 10	0.5020	0.4985
<i>max_depth</i> = 12	0.5020	0.4983

The optimal hyperparameter for the decision tree on this dataset is *max_depth* = 10.

2. Random Forest:

Hyperparameter (# of Trees)	Accuracy	F1-Score
<i>trees</i> = 1	0.4420	0.4429
<i>trees</i> = 5	0.5520	0.5438
<i>trees</i> = 10	0.6130	0.6092
<i>trees</i> = 20	0.6700	0.6681
<i>trees</i> = 30	0.6980	0.6925
<i>trees</i> = 40	0.7020	0.6957

The optimal hyperparameter for a Random Forest on this dataset is *trees* = 30.

3. Neural Network (Architecture: [40, 40, 10], Alpha: 0.1, Lambda: 0.01):

Hyperparameter (Epochs)	Accuracy	F1-Score
<i>epochs</i> = 50	0.6580	0.6471
<i>epochs</i> = 100	0.7070	0.7020
<i>epochs</i> = 250	0.7310	0.7269
<i>epochs</i> = 500	0.7440	0.7410
<i>epochs</i> = 1000	0.7380	0.7339
<i>epochs</i> = 2000	0.7380	0.7342

The optimal hyperparameter for the Neural Network is *epochs* = 500.

1.5.2 Learning Curves and Analysis

Below are the learning curves for the three evaluated algorithms.

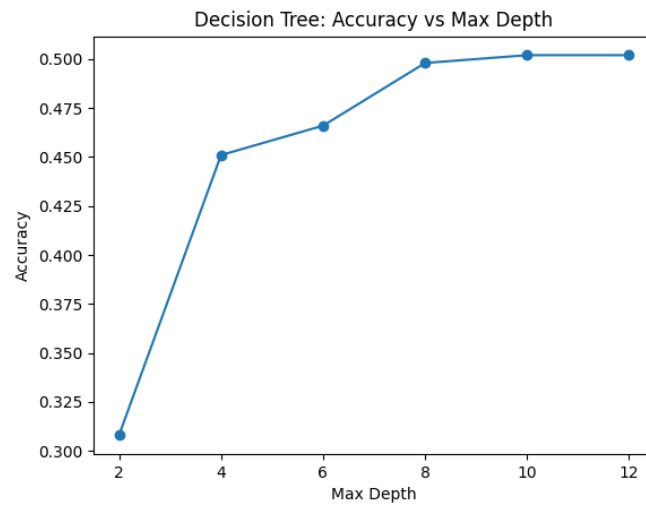


Figure 17: Decision Tree Learning Curve: Accuracy vs. *max_depth*

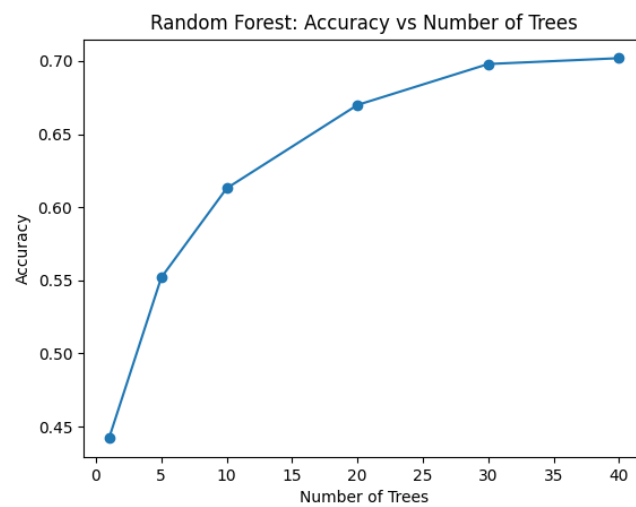


Figure 18: Random Forest Learning Curve: Accuracy vs. Number of Trees

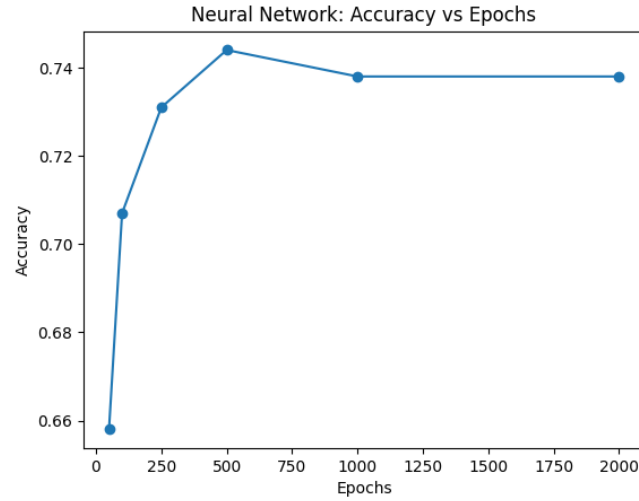


Figure 19: Neural Network Learning Curve: Accuracy vs. Epochs

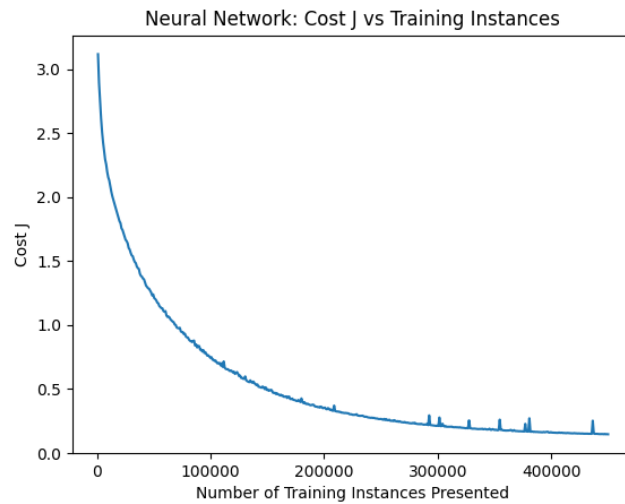


Figure 20: Neural Network Learning Curve: Cost J vs. Training Instances Presented

Analysis and Interpretation: As expected, the neural network by far had the highest performance, with an optimized accuracy of 74.4% at 500 epochs. We can see that we pushed the cost function pretty low with this configuration, so we can say it is relatively optimized.

On the other hand, our decision tree did not perform nearly as well, but still was able to make some decent progress with an optimized accuracy of 50.2% at $max_depth = 10$. Considering there are 10 possible labels, a random guess would have an accuracy of 10%, so this is way better than random, but still not at the level of the neural net.

The random forest performed much better than the decision tree, with an optimized accuracy of 70.2% at $n_tree = 40$. Again, with a higher n_tree value we may have been able to squeeze just a little bit more performance out of it, but the algorithm was already taking an extremely long time to train and we can see the accuracy plateau strongly already.

Overall though, the neural network had the best performance as it was able to learn the complex relationships in the mathematical data.

1.6 [Extra Credit] Hybrid kNN-Decision-Tree Ensemble

Task Assigned To: Max Goldman

Implementations Used: Max's implementation of Decision Tree and Random Forest, Peter's implementation of kNN.

Algorithm Explanation: This hybrid ensemble consists of equal parts kNN and decision tree. For each individual kNN "vote" in the ensemble, the kNN algorithm is run on a corresponding bootstrapped dataset. Decision tree votes work the same as in a random forest. After every member of the ensemble has voted, either the majority wins, or a random choice is made if there is a tie.

1.6.1 Hyperparameter Tuning

The chosen hyperparameter for this ensemble is n , which represents the number of trees and the number of kNNs. That is, there will be n trees and n kNNs evaluated, so there will be $2n$ votes total. The k value for the kNNs is held at 3, and 6 different hyperparameter settings were used for each dataset. The performance of the algorithm was evaluated using Stratified 10-Fold Cross-Validation, measuring both Accuracy and F1-Score (Weighted). The results averaged across the 10 folds are shown below:

1. Parkinson's Dataset:

Hyperparameter (n)	Accuracy	F1-Score
$n = 1$	0.8045	0.7658
$n = 3$	0.8408	0.8013
$n = 5$	0.8816	0.8607
$n = 7$	0.8255	0.7806
$n = 9$	0.8100	0.7621
$n = 11$	0.8008	0.7408

The optimal hyperparameter for this on this dataset is $n = 5$.

2. Rice Dataset:

Hyperparameter (n)	Accuracy	F1-Score
$n = 1$	0.9134	0.9135
$n = 2$	0.9147	0.9146
$n = 3$	0.9173	0.9174
$n = 4$	0.9344	0.9344
$n = 5$	0.9199	0.9199
$n = 6$	0.9370	0.9370

The optimal hyperparameter for this on this dataset is $n = 6$.

1.6.2 Learning Curves and Analysis

Shown below is the learning curve of this hybrid ensemble as evaluated on the Parkinson's dataset (1.2) and Rice dataset (1.3).

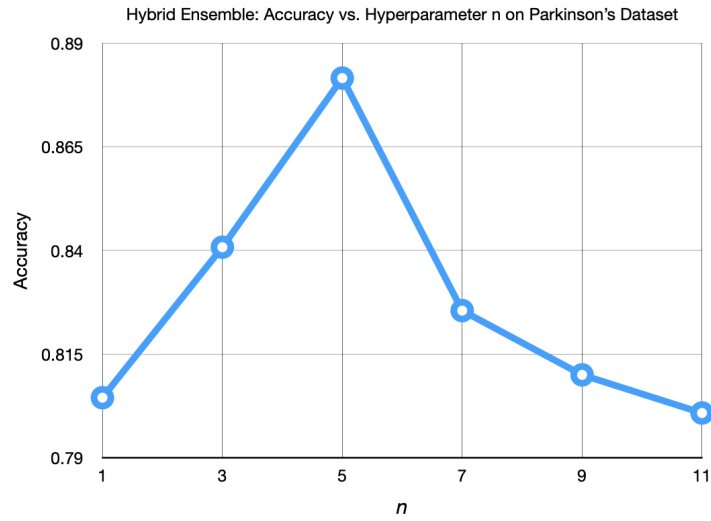


Figure 21: Learning Curve on Parkinson's Dataset: Accuracy vs. Hyperparameter n

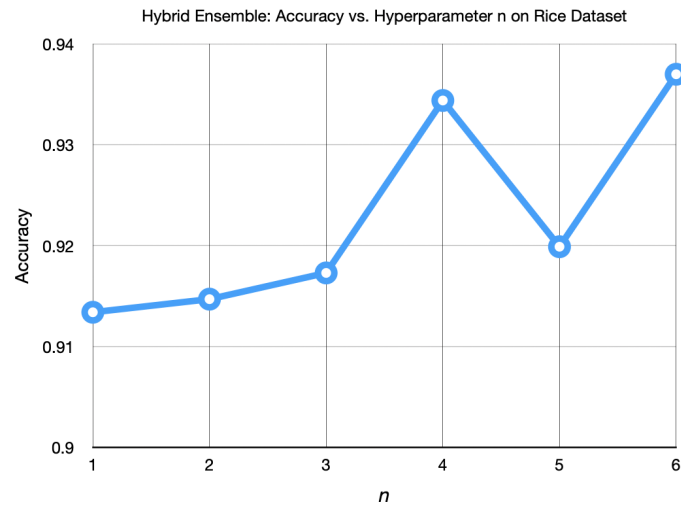


Figure 22: Learning Curve on Parkinson's Dataset: Accuracy vs. Hyperparameter n

Analysis and Interpretation:

We can see in the performance on both datasets that the accuracy increases as n increases to 4 or 5. This is because the variance from bootstrapping along with ensembling voting is allowing the algorithm to reach better conclusions. However, if we keep increasing n , the number of bootstrapped datasets increases to a point where accuracy begins decreasing. This is due to the fact that the variance from bootstrapping begins leading to inaccuracy on the kNN datasets, and thus drags our accuracy down.

1.7 Overall Results Summary

The table below summarizes the best performance achieved by each algorithm on the datasets.

	Dataset 1		Dataset 2		Dataset 3		Dataset 4		Dataset 5	
	Accuracy	F1-Score	Accuracy	F1-Score	Accuracy	F1-Score	Accuracy	F1-Score	Accuracy	F1-Score
k-NN	0.9894	0.9894	0.9630	0.9290	0.9250	0.9350				
Decision Tree			0.8445	0.7898	0.9239	0.9223	0.8637	0.8636	0.5020	0.4985
Random Forest	0.9722	0.9721	0.9240	0.9220	0.9252	0.9252	0.8591	0.8591	0.7020	0.6957
Neural Network	0.9750	0.9749					0.8652	0.8652	0.7440	0.7410
kNN-DT Ensemble			0.8816	0.8607	0.9370	0.9370				

Table 1: Overall Performance Summary Table